TaxonVision-MLOps

Comprehensive Technical Implementation Guide

Systems Architecture, Infrastructure, and Codebase Reference

Benjamin Förster

October 11, 2026

Abstract

This technical report serves as an exhaustive, implementation-level architectural manual and code base reference for the TaxonVision-MLOps platform. Designed to document every structural component, configuration file, deployment manifest, mathematical loss formulation, and runtime daemon in the active repository, this document provides complete visibility into the physical server topology (`hive-mind` running Ubuntu 26.04.1 LTS, NVIDIA Driver 615.71.09, CUDA 13.4, and the NVIDIA GeForce RTX 5070 Ti with Blackwell SM 120), the lightweight Kubernetes (k3s) cluster partitioned into `jenkins-ci` and `taxon-vision` namespaces, the Cloudflare Zero Trust ingress tunnel, and the four-slot NVIDIA GPU time-slicing device plugin. Furthermore, this manual dissects the pure Pixi multi-environment toolchain, the automated DagsHub storage remote for Data Version Control (DVC) and MLflow tracking, the Cui et al. Class-Balanced Loss dynamics on biological long-tail Pareto distributions, the Optuna Bayesian hyperparameter optimisation engine, Split Conformal Prediction, Energy-Based Out-of-Distribution rejection, and the unified FastAPI and HTMX operator dashboard. Every configuration parameter, file extension, and container specification is strictly analysed based upon active codebase evidence.

Contents

Abstract i

1 Repository Architecture, Directory Hierarchy, and File Extension Semantics 1
- 1.1 Root Directory Topology . . . 1
- 1.2 Configuration Manifests and Semantics . . . 1
- 1.3 Specialised File Formats and Storage Conventions . . . 1

2 Bare-Metal Server Environment and Pixi Toolchain Interlock 2
- 2.1 Physical Compute Node Baseline . . . 2
- 2.2 Multi-Target Toolchain and Pixi Environment Isolation . . . 2
- 2.3 Command Orchestration via Justfile . . . 2

3 Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC 3
- 3.1 Cluster Architecture and Network Edge . . . 3
- 3.2 Namespace Partitioning and Multi-Tenant Segregation . . . 3
- 3.3 Role-Based Access Control (RBAC) Specification . . . 3

4 GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas 4
- 4.1 The Single-GPU Multi-Tenancy Challenge . . . 4
- 4.2 NVIDIA Device Plugin Time-Slicing Configuration . . . 4
- 4.3 Deterministic VRAM Budget Capping and OOM Prevention . . . 4

5 Containerisation Architecture: Dockerfile and Composition Manifests 5
- 5.1 Multi-Stage Container Construction Strategy . . . 5
- 5.2 Auxiliary Services and Docker Compose Topology . . . 5
- 5.3 Container Hardening and Privilege Minimisation . . . 5

6 Kubernetes Workloads, Service Manifests, and Rolling Updates 6
- 6.1 API Deployment Specification . . . 6
- 6.2 Health Probes and Zero-Downtime Rolling Restarts . . . 6
- 6.3 Kubernetes Service Abstraction and Routing . . . 6

7 Declarative Jenkins Continuous Integration and Ephemeral Kubernetes Agents 7
- 7.1 Dynamic Ephemeral Pod Lifecycle in Kubernetes . . . 7
  - 7.1.1 Security Context and Role-Based Access Control . . . 8
  - 7.1.2 Containerisation Strategy and Resource Allocation . . . 8
  - 7.1.3 Package Caching Strategy . . . 8
- 7.2 Credential Management and Cloud Storage Authentication . . . 9
- 7.3 Stage-by-Stage Pipeline Architecture . . . 9
  - 7.3.1 Stage 1: Prepare Toolchain . . . 9
  - 7.3.2 Stage 2: Static Quality and Invariants . . . 10
  - 7.3.3 Stage 3: Unit Tests and Coverage . . . 10
  - 7.3.4 Stage 4: Integration and Conformal Invariants . . . 10
  - 7.3.5 Stage 5: Documentation Strict Build . . . 10

7.3.6 Stage 6: Model Training and Checkpoint (GPU) 10
7.3.7 Stage 7: Deploy to Kubernetes 11
7.4 Resilience, Cleanup, and Pipeline Options 11

8 Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits 12
8.1 iNaturalist API Ingestion and Query Mechanics 12
8.2 Zero-Copy Apache Arrow Streaming Pipeline 12
8.3 Open License Verification and Attribution Retention 12

9 Data Version Control (DVC) and Remote Storage Topology 13
9.1 DVC Pipeline Graph and Cryptographic Manifests 13
9.2 DagsHub S3-Compatible Remote Storage Architecture 13
9.3 Synchronization Lifecycles in CI/CD and Local Workstations 13

10 Domain Modelling, DarwinCore Standards, and Taxonomic Trees 14
10.1 DarwinCore Taxon Schema and Catalog Implementation 14
10.2 Observation Entities and Spatiotemporal Metadata 14
10.3 License Verification Domain Invariants 14

11 Vision Backbone Factory and Representation Extraction Caching 15
11.1 Foundation Vision Backbone Portfolio 15
11.2 Extensible Feature Extractor Factory Architecture 15
11.3 Representation Extraction and Disk-Caching Strategy 15

12 Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics 16
12.1 The Biological Long-Tail Pareto Distribution 16
12.2 Effective Number of Samples Mathematical Derivation 16
12.3 PyTorch Implementation and Numerical Stabilisation 16

13 Classification Head Architecture, Optimization Loops, and Callbacks 17
13.1 Classification Head Geometry and Transfer Learning 17
13.2 Cached Embedding Training Loop and Mini-Batch Dynamics 17
13.3 Convergence Callbacks and Best-Weight Restoration 17

14 Bayesian Hyperparameter Optimization with Optuna TPE and ASHA 18
14.1 Bayesian Optimization Principles and TPE Mechanics 18
14.2 Search Space Parameterisation and Objective Formulation 18
14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice 18

15 Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry 19
15.1 Experiment Tracking Architecture on DagsHub 19
15.2 Deterministic Lifecycle Management via RAII Scopes 19
15.3 Cryptographic Lineage Capture and Automated Model Promotion 19

16 Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection 20
16.1 The Epistemic Safety Problem in Biological Vision 20
16.2 Split Conformal Prediction and Statistical Coverage Bounds 20
16.3 Helmholtz Free Energy Out-of-Distribution Detection 20

17 Active Learning Strategy and Priority Triage Queue 21
17.1 Uncertainty-Based Query Acquisition Mechanics 21
17.2 Priority Triage Queue Implementation 21
17.3 Closing the Continuous Retraining Annotation Loop 21

18 ONNX Export, Zero-Allocation Runtime, and INT8 Quantization 22
18.1 Computational Graph Export and Dynamic Axes 22
18.2 Zero-Allocation C++ Inference Execution Engine 22

18.3 INT8 Quantization Calibration and Class Activation Mapping 22

19 FastAPI REST Service and Server-Rendered HTMX Operator Dashboard 23
19.1 Unified REST and Web Application Architecture 23
19.2 API Schemas, Endpoints, and Inference Routing 23
19.3 Server-Rendered HTMX Operator Interface 23

20 Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection 24
20.1 Prometheus Telemetry Instrumentation and Metrics Design 24
20.2 Serving Latency and Conformal Set Monitoring 24
20.3 Embedding Representation Drift via Wasserstein and MMD 24

21 Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets 25
21.1 Testing Architecture and Test Matrix Topology 25
21.2 Mathematical Invariant Assertions and Property-Based Testing 25
21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates 25

22 Open Knowledge Format (OKF v0.2) and Master Agent Routing 26
22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers 26
22.2 Multi-Agent Governance and Decision Routing Matrices 26
22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization 26

23 Developer Operations, Environment Setup, and Engineering Runbooks 27
23.1 Developer Workstation Onboarding and Environment Setup 27
23.2 Dataset Synchronization and Remote Storage Runbook 27
23.3 Cluster Operations, GPU Debugging, and Emergency Rollback 27

Chapter 1

Repository Architecture, Directory Hierarchy, and File Extension Semantics

1.1 Root Directory Topology

1.2 Configuration Manifests and Semantics

1.3 Specialised File Formats and Storage Conventions

Chapter 2

Bare-Metal Server Environment and Pixi Toolchain Interlock

2.1 Physical Compute Node Baseline

2.2 Multi-Target Toolchain and Pixi Environment Isolation

2.3 Command Orchestration via Justfile

Chapter 3

Lightweight Kubernetes (k3s) Cluster Topology and Security RBAC

3.1 Cluster Architecture and Network Edge

3.2 Namespace Partitioning and Multi-Tenant Segregation

3.3 Role-Based Access Control (RBAC) Specification

Chapter 4

GPU Virtualisation, NVIDIA Device Plugin, and VRAM Quotas

4.1 The Single-GPU Multi-Tenancy Challenge

4.2 NVIDIA Device Plugin Time-Slicing Configuration

4.3 Deterministic VRAM Budget Capping and OOM Prevention

Chapter 5

Containerisation Architecture: Dockerfile and Composition Manifests

5.1 Multi-Stage Container Construction Strategy

5.2 Auxiliary Services and Docker Compose Topology

5.3 Container Hardening and Privilege Minimisation

Chapter 6

Kubernetes Workloads, Service Manifests, and Rolling Updates

6.1 API Deployment Specification

6.2 Health Probes and Zero-Downtime Rolling Restarts

6.3 Kubernetes Service Abstraction and Routing

Chapter 7

Declarative Jenkins Continuous Integration and Ephemeral Kubernetes Agents

The continuous integration and continuous deployment (CI/CD) methodology constitutes the fundamental circulatory system of the machine learning operations lifecycle. In the context of the TaxonVision-MLOps platform, manual execution of quality assurance testing, dependency resolution, hyperparameter optimisation, and deployment manifest application is explicitly forbidden. Instead, the platform relies upon a strictly declarative Jenkins pipeline architecture, which dynamically provisions ephemeral, isolated compute environments within a managed Kubernetes cluster. This chapter details the comprehensive technical implementation of this architecture, strictly mapping every theoretical concept to its concrete manifestation within the repository's `Jenkinsfile` and the associated Kubernetes Role-Based Access Control (RBAC) definitions located in `deploy/k8s/jenkins-agent-rbac.yaml`. The ensuing exposition delineates the dynamic lifecycle of ephemeral pod agents, the rigorous cryptographic isolation of cloud storage credentials, the sequential pipeline execution matrix, and the ultimate cleanup resilience mechanics. The rigorous standardisation achieved through this declarative paradigm ensures a high degree of automation and repeatability across all stages of the software development lifecycle, thus minimising the probability of human error during complex integration manoeuvres.

7.1 Dynamic Ephemeral Pod Lifecycle in Kubernetes

The foundational principle of the TaxonVision-MLOps deployment strategy is the eradication of persistent, mutable execution states. Traditional static CI runners accumulate latent configurations and residual artefacts over successive executions, leading to deterministic failure divergence, commonly termed "configuration drift". To neutralise this vulnerability, the Jenkins architecture is integrated with the Kubernetes agent plugin, dynamically provisioning a pristine execution environment for every invocation of the pipeline. This approach intrinsically aligns with the principles of immutable infrastructure, whereby the execution context is instantiated entirely from a deterministic specification rather than being modified in place.

The agent declaration within the `Jenkinsfile` explicitly targets the `k3s` cloud provider. This integration is formally declared as follows, specifying the deployment within the isolated `jenkins-ci` namespace:

```
pipeline {
  agent {
    kubernetes {
```

```
      cloud 'k3s'
      defaultContainer 'ml-runner'
      yaml '''
apiVersion: v1
kind: Pod
metadata:
  namespace: jenkins-ci
  labels:
    app.kubernetes.io/name: taxon-vision-jenkins-agent
spec:
  serviceAccountName: jenkins-agent
  ...
      '''
    }
  }
}
```

7.1.1 Security Context and Role-Based Access Control

The execution of privileged actions, particularly deploying modifications to the production cluster environment, necessitates a meticulously scoped security context. The ephemeral pod is assigned the service account `jenkins-agent`, residing within the `jenkins-ci` namespace. The configuration defined in `deploy/k8s/jenkins-agent-rbac.yaml` establishes a robust principle of least privilege.

Within the `jenkins-ci` namespace, the agent is granted restricted capabilities to manage its own execution logs and execution state. Furthermore, a secondary RoleBinding associates the `jenkins-agent` service account with the `jenkins-deployer-role` in the target `taxon-vision` namespace. This cross-namespace authorisation strictly confines the agent's mutability vectors to the management of `deployments`, `services`, and `pods` required for the final zero-downtime deployment phase, thereby mitigating the risk of systemic cluster compromise. This deliberate segmentation prevents a compromised CI agent from escalating its privileges across the wider Kubernetes cluster architecture.

7.1.2 Containerisation Strategy and Resource Allocation

The workload is executed within a unified container environment named `ml-runner`. Crucially, the image is pinned to the minimal `ghcr.io/prefix-dev/pixi:latest`, guaranteeing an environment pre-equipped with the Pixi multi-environment package manager. To prevent premature termination whilst the Jenkins controller orchestrates the stages, the container is initiated with a persistent sleep command (`command: ["sleep"]`, `args: ["99d"]`).

Machine learning workloads impose extreme demands upon the computational substrate. The ephemeral pod specification strictly codifies resource requests and limits. The container is guaranteed a baseline allocation of 512Mi memory and 200m CPU. However, to accommodate the intensive requirements of Bayesian hyperparameter tuning and model compilation, it is permitted to burst to limits of 8Gi memory and 4 CPU cores. Most significantly, the specification mandates the provisioning of a hardware accelerator via the NVIDIA device plugin: `nvidia.com/gpu: "1"`. This critical declaration ensures the pod is scheduled exclusively onto a node possessing an available time-sliced GPU partition, an absolute prerequisite for executing the stage responsible for the classification head training. The orchestration of these hardware resources is non-negotiable for achieving acceptable latency during the Bayesian optimisation phase.

7.1.3 Package Caching Strategy

To prevent catastrophic latency amplification during dependency resolution, a strategic caching mechanism is implemented. An `emptyDir: {}` volume named `rattler-cache` is mounted at `/root/.cache/rattler/cache` within the container filesystem. This directory caches the downloaded Conda and PyPI packages handled by the underlying rattler library. While the volume is ephemeral to the pod's lifespan, it persists across the parallel and sequential execution stages of a

single pipeline run, drastically accelerating the repeated `pixi install` invocations occurring in disparate stages. This strategic use of local caching mitigates the network overhead associated with repeatedly pulling voluminous software packages from external registries.

7.2 Credential Management and Cloud Storage Authentication

The necessity for continuous integration systems to interact with external services, specifically the DagsHub remote storage and MLflow tracking server, introduces severe credential management vulnerabilities. The pipeline explicitly rejects the anti-pattern of embedding cryptographic secrets within container images or configuration files. Instead, it relies upon Jenkins' internal credential store and environmental injection.

The pipeline environment block defines a pristine isolation matrix. A single administrative secret, identified within Jenkins as `dagshub-token`, is retrieved and injected dynamically into the ephemeral execution context.

```
environment {
  DAGSHUB_TOKEN = credentials('dagshub-token')
  AWS_ACCESS_KEY_ID = "${DAGSHUB_TOKEN}"
  AWS_SECRET_ACCESS_KEY = "${DAGSHUB_TOKEN}"
  MLFLOW_TRACKING_USERNAME = 'foersben'
  MLFLOW_TRACKING_PASSWORD = "${DAGSHUB_TOKEN}"
}
```

This token is subsequently multiplexed to satisfy disparate authentication protocols. For the Data Version Control (DVC) S3-compatible remote storage endpoint, the token is mapped onto the standard `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables. Simultaneously, it serves as the `MLFLOW_TRACKING_PASSWORD` for authenticating experimental telemetry streams to the MLflow server. This architecture guarantees that the tokens remain strictly as in-memory environment variables, confined to the lifespan of the running pod, and are never written to physical disk or logged into standard output streams. This rigorous approach fundamentally secures the platform against token exfiltration vulnerabilities that commonly afflict poorly architected CI pipelines.

7.3 Stage-by-Stage Pipeline Architecture

The pipeline executes a rigorous sequence of quality gates, culminating in model training and deployment. This deterministic directed acyclic graph ensures that code regressions, mathematical invalidations, and structural anomalies are intercepted long before they contaminate the production runtime environment. The stages are architected to fail fast, thereby providing immediate feedback to the engineering team and halting the progression of fundamentally flawed modifications.

7.3.1 Stage 1: Prepare Toolchain

The initial stage prepares the operational substrate. Within the `ml-runner` container, the pipeline dynamically provisions necessary external clients, specifically `git` and `kubernetes-client`, ensuring absolute standardisation across all potential base node configurations. The core project environment is then materialised using `pixi install -frozen -e ci-dev`, ensuring deterministic dependency resolution based strictly upon the `pixi.lock` file. Finally, the stage synchronises the multi-gigabyte dataset and model assets from the DagsHub remote using `dvc pull`, establishing the complete context for subsequent testing and training. This deterministic initialisation guarantees that subsequent stages operate upon a mathematically defined known-good state.

7.3.2 Stage 2: Static Quality and Invariants

To maximise computational throughput, static analysis is parallelised into three concurrent execution branches.

- **Lint and Format:** The Ruff linter and formatter (`ruff check .` and `ruff format -check .`) enforce absolute syntactic consistency and stylistic compliance.
- **Type Checking:** Mypy (`mypy src scripts`) statically verifies the type integrity of the entire codebase, preventing subtle runtime type errors that are particularly pernicious in complex tensor manipulations.
- **Compliance and OKF Audits:** Custom analytical scripts are executed to ensure holistic repository integrity. The `audit_license_compliance.py` script guarantees the absence of incompatible open-source licenses, whilst `validate_okf.py` enforces rigorous adherence to the Open Knowledge Format standard for documentation metadata.

The concurrent execution of these static gates minimises the overall pipeline duration, allowing the immediate identification of fundamental syntactic or logical errors prior to the commencement of computationally expensive testing phases.

7.3.3 Stage 3: Unit Tests and Coverage

The unit testing phase executes the Pytest suite against the core source directory. The crucial mechanism here is the enforcement of a strict coverage threshold via the `-cov-fail-under=78` argument. Any codebase modification that depresses the test coverage metric below 78% results in an immediate, deterministic pipeline failure, mechanically enforcing a culture of comprehensive test authoring. This threshold acts as an objective barrier against the accretion of untested and therefore unverified logical pathways within the core system.

7.3.4 Stage 4: Integration and Conformal Invariants

This stage evaluates the structural integrity of the integration pathways and enforces the mathematical assertions underlying the uncertainty estimation framework. Executing with the `-x -q` flags ensures early exit upon the first encountered failure, immediately terminating the pipeline in the event of an invariant violation. This rigorous gatekeeping guarantees that the Split Conformal Prediction algorithms and the energy-based out-of-distribution (OOD) rejection thresholds operate within their mathematically defined bounds. The validation of these invariants is non-negotiable for ensuring the platform's epistemic reliability and generalisability.

7.3.5 Stage 5: Documentation Strict Build

The integrity of the knowledge base is verified through a strict build process. The pipeline executes `visualize_okf.py` to dynamically generate accurate graph representations of the Open Knowledge Format metadata. Subsequently, `zensical build -strict` is executed, generating the final documentation artefacts. The `-strict` flag guarantees that any broken reference, malformed LaTeX equation, or syntax violation within the documentation immediately fails the pipeline, preserving the repository as a pristine, error-free encyclopaedia. The objective is to treat technical documentation with the identical level of stringency as the executable codebase.

7.3.6 Stage 6: Model Training and Checkpoint (GPU)

This advanced stage operates conditionally, triggering exclusively upon commits merged into the `main` or `feature/phase1-foundations` branches. It instantiates the full `dev` environment and initiates the Bayesian hyperparameter tuning sequence using Optuna. The training script is invoked with parameters enforcing a rigorous optimisation protocol: `-tune -n-trials 10 -cuda-memory-fraction 0.7`. This dictates the execution of ten separate trials, strictly limiting the GPU VRAM allocation to 70% to prevent out-of-memory cascading failures on the time-sliced GPU partition.

Upon identification of the optimal configuration, the pipeline executes the `onnx_exporter` to serialise the resultant computational graph into the highly optimised ONNX format. The stage concludes by pushing both the updated PyTorch weights (`head.pt`) and the accelerated ONNX artifact (`model.onnx`) back to the DagsHub remote via `dvc push`, thereby memorialising the specific algorithmic outcome of the integration pipeline. The explicit tracking of these binaries guarantees the complete reproducibility of the derived intelligence.

7.3.7 Stage 7: Deploy to Kubernetes

The final execution phase authorises the rollout to the production cluster. The pipeline applies the Kubernetes manifest using `kubectl apply -f deploy/k8s/api-deployment.yaml`. The critical component of this deployment is the subsequent `kubectl rollout restart deployment/taxon-vision-api -n taxon-vision` command, which triggers a zero-downtime rolling update. The pipeline continuously monitors the rollout status, imposing a rigorous 120-second timeout (`-timeout=120s`). Should the new pods fail to achieve readiness within this window, the rollout is flagged as failed, preserving the stability of the antecedent deployment iteration. This automated orchestration eliminates the manual intervention previously required to propagate novel machine learning capabilities into the active production environment.

7.4 Resilience, Cleanup, and Pipeline Options

Systemic resilience encompasses not merely successful execution, but also rigorous cleanup and constraint management. The Jenkins pipeline is heavily instrumented with configuration options designed to maintain pristine operational environments and prevent runaway resource consumption. The enforcement of these administrative constraints is essential for maintaining the long-term viability of the shared execution infrastructure.

The global `options` block implements a stringent garbage collection policy. The `buildDiscarder(logRotator(numToKe ’15’, daysToKeepStr: ’14’))` directive limits historical log retention to fifteen builds or a maximum of fourteen days, ensuring the Jenkins master node filesystem is not exhausted by extensive log accumulation. A global execution timeout of forty-five minutes (`timeout(time: 45, unit: ’MINUTES’)`) prevents rogue or deadlocked processes from indefinitely occupying the ephemeral compute nodes.

Furthermore, the `disableConcurrentBuilds(abortPrevious: true)` directive ensures a linear execution topology. If a rapid succession of commits is pushed, previous concurrent runs are immediately aborted, conserving computational bandwidth for the most recent and relevant execution context. This policy optimises resource utilisation by preemptively terminating computations that have been effectively superseded by subsequent modifications.

Finally, irrespective of pipeline success or failure, a rigorous `post { always { cleanWs() } }` block is executed. This directive enforces absolute workspace hygiene, guaranteeing that no residual data files, cached matrices, or intermediate compilation artefacts persist within the ephemeral container’s filesystem. While the pod itself is ultimately terminated by Kubernetes, this explicit workspace deletion guarantees a sanitised teardown sequence, fundamentally preserving the stateless architecture of the CI/CD integration process. The orchestration of these discrete mechanisms results in a mathematically predictable, reproducible, and fault-tolerant release engineering pipeline.

Chapter 8

Data Ingestion Engine, Zero-Copy Arrow Streaming, and License Audits

8.1 iNaturalist API Ingestion and Query Mechanics

8.2 Zero-Copy Apache Arrow Streaming Pipeline

8.3 Open License Verification and Attribution Retention

Chapter 9

Data Version Control (DVC) and Remote Storage Topology

9.1 DVC Pipeline Graph and Cryptographic Manifests

9.2 DagsHub S3-Compatible Remote Storage Architecture

9.3 Synchronization Lifecycles in CI/CD and Local Workstations

Chapter 10

Domain Modelling, DarwinCore Standards, and Taxonomic Trees

10.1 DarwinCore Taxon Schema and Catalog Implementation

10.2 Observation Entities and Spatiotemporal Metadata

10.3 License Verification Domain Invariants

Chapter 11

Vision Backbone Factory and Representation Extraction Caching

11.1 Foundation Vision Backbone Portfolio

11.2 Extensible Feature Extractor Factory Architecture

11.3 Representation Extraction and Disk-Caching Strategy

Chapter 12

Class-Balanced Loss Formulation and Long-Tail Gradient Dynamics

12.1 The Biological Long-Tail Pareto Distribution

12.2 Effective Number of Samples Mathematical Derivation

12.3 PyTorch Implementation and Numerical Stabilisation

Chapter 13

Classification Head Architecture, Optimization Loops, and Callbacks

13.1 Classification Head Geometry and Transfer Learning

13.2 Cached Embedding Training Loop and Mini-Batch Dynamics

13.3 Convergence Callbacks and Best-Weight Restoration

Chapter 14

Bayesian Hyperparameter Optimization with Optuna TPE and ASHA

14.1 Bayesian Optimization Principles and TPE Mechanics

14.2 Search Space Parameterisation and Objective Formulation

14.3 Asynchronous Successive Halving (ASHA) Pruning in Practice

Chapter 15

Experiment Tracking, Cryptographic Lineage, and MLflow Model Registry

15.1 Experiment Tracking Architecture on DagsHub

15.2 Deterministic Lifecycle Management via RAII Scopes

15.3 Cryptographic Lineage Capture and Automated Model Promotion

Chapter 16

Epistemic Safety: Split Conformal Prediction and Energy-Based OOD Rejection

16.1 The Epistemic Safety Problem in Biological Vision

16.2 Split Conformal Prediction and Statistical Coverage Bounds

16.3 Helmholtz Free Energy Out-of-Distribution Detection

Chapter 17

Active Learning Strategy and Priority Triage Queue

17.1 Uncertainty-Based Query Acquisition Mechanics

17.2 Priority Triage Queue Implementation

17.3 Closing the Continuous Retraining Annotation Loop

Chapter 18

ONNX Export, Zero-Allocation Runtime, and INT8 Quantization

18.1 Computational Graph Export and Dynamic Axes

18.2 Zero-Allocation C++ Inference Execution Engine

18.3 INT8 Quantization Calibration and Class Activation Mapping

Chapter 19

FastAPI REST Service and Server-Rendered HTMX Operator Dashboard

19.1 Unified REST and Web Application Architecture

19.2 API Schemas, Endpoints, and Inference Routing

19.3 Server-Rendered HTMX Operator Interface

Chapter 20

Real-Time Telemetry, Prometheus Instrumentation, and Drift Detection

20.1 Prometheus Telemetry Instrumentation and Metrics Design

20.2 Serving Latency and Conformal Set Monitoring

20.3 Embedding Representation Drift via Wasserstein and MMD

Chapter 21

Quality Assurance: Pytest Matrix, Invariant Testing, and Complexity Budgets

21.1 Testing Architecture and Test Matrix Topology

21.2 Mathematical Invariant Assertions and Property-Based Testing

21.3 Cognitive Complexity Budgets and Pre-Commit Quality Gates

Chapter 22

Open Knowledge Format (OKF v0.2) and Master Agent Routing

22.1 Open Knowledge Format (OKF v0.2) Schema and Trust Tiers

22.2 Multi-Agent Governance and Decision Routing Matrices

22.3 Automated Validation Scripts and D3 Knowledge Graph Visualization

Chapter 23

Developer Operations, Environment Setup, and Engineering Runbooks

23.1 Developer Workstation Onboarding and Environment Setup

23.2 Dataset Synchronization and Remote Storage Runbook

23.3 Cluster Operations, GPU Debugging, and Emergency Rollback

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.